

Exploring Adversarial Attacks on the MaSTer Truncation Protocol

Martin Zbudila
COSIC, KU Leuven
Leuven, Belgium
martin.zbudila@esat.kuleuven.be

Aysajan Abidin
COSIC, KU Leuven
Leuven, Belgium
aysajan.abidin@esat.kuleuven.be

Bart Preneel
COSIC, KU Leuven
Leuven, Belgium
bart.preneel@esat.kuleuven.be

Abstract

At CANS 2024, Zbudila et al. presented MaSTer, a maliciously secure multi-party computation protocol for truncation. It allows adversaries to manipulate outputs with a bounded additive error while avoiding detection with a certain probability. In this work, we analyse the broader implications of adversarial exploitation in probabilistic truncation protocols, specifically in relation to MaSTer. We propose three attack strategies aimed at inducing misclassification in deep neural network (DNN) inference. Our empirical evaluation across multiple datasets demonstrates that while adversarial influence remains negligible under realistic constraints, certain configurations and network architectures exhibit increased vulnerability. By improving the understanding of the risks associated with probabilistic truncation protocols in privacy-preserving machine learning, our work demonstrates that the MaSTer protocol is robust in realistic settings.

Keywords

Multi Party Computation, Machine Learning, Attack, Truncation

1 Introduction

The wide deployment of machine learning brings substantial privacy risks. There is a growing interest in developing and deploying Privacy-Preserving Machine Learning (PPML) based on secure Multi-Party Computation (MPC). It aims to enable collaborative model training and inference while preserving the confidentiality of sensitive data. Several MPC frameworks have been developed to facilitate the secure and efficient evaluation of neural networks, leveraging specialized cryptographic techniques to perform computations over encrypted or secret-shared data [3, 13, 16, 17, 22]. These frameworks rely heavily on fixed-point arithmetic for efficiency, as floating-point operations introduce significant computational and communication overhead. However, the reliance on fixed-point arithmetic necessitates the use of truncation, which is an integral part of secure computation.

Truncation, the process of reducing precision in fixed-point representations, plays a crucial role in PPML frameworks as it directly impacts computational and communication efficiency. Given its significance, truncation protocols have garnered substantial attention from the research community. In CANS 2024, Zbudila et al. [24] introduced MaSTer, a maliciously secure truncation protocol, which introduces a new adversarial setting wherein a corrupt party can influence the output by, at most, an additive error of $|\Delta| \leq 2$, with non-negligible probability. Here, Δ denotes the additive error that an adversary can maliciously introduce to the secret-shared value.

In this work, we systematically analyse the implications of such perturbations when applied iteratively throughout a computational circuit. Intuitively, the deeper the circuit, the greater the number of multiplications involved, thereby amplifying the cumulative effect of even minor adversarial modifications. Given that machine learning—particularly deep neural networks—represents one of the most computationally intensive applications of secure computation, these architectures provide a compelling setting for our analysis.

Our study focuses on deep neural networks composed of fully connected layers, as these architectures exhibit substantial multiplicative depth, making them particularly susceptible to adversarial perturbations. We investigate various attack strategies that exploit iterative perturbations at each truncation step. Our experiments evaluate these novel attacks on neural network inference using MaSTer’s truncation vulnerability. However, under realistic adversarial constraints, the impact remains negligible, often leading to frequent protocol aborts. This highlights the robustness of MaSTer for real-world deployments, as its security properties effectively mitigate adversarial influence under realistic assumptions on the adversary.

Contributions. In the following we list our main contributions.

- We provide a deeper analysis of malicious behaviour within the MaSTer protocol, specifically evaluating the probability of detection when an adversary attempts to introduce a controlled additive error of $\Delta = \pm 1$.
- We propose three distinct attack strategies designed to misclassify samples during neural network inference. These strategies operate within realistic adversarial constraints, requiring no additional assumptions on the adversary’s knowledge or computational power.
- We implement our attack strategies on a range of neural network architectures using five different machine learning datasets. Our experiments vary key parameters, such as the adversary’s capabilities, fixed-point arithmetic configurations, and optimisations related to the truncation protocol, to evaluate the feasibility of the attack under different settings.
- We thoroughly analyse our experimental results, assessing the practical feasibility of these attacks and their implications for deploying MaSTer in real-world PPML applications. Our findings contribute to a better understanding of the trade-offs between efficiency and security in maliciously secure truncation protocols.

2 Preliminaries

In Section 2.1, we provide an overview of multi-party computation (MPC), while Section 2.2 introduces the fundamentals of neural networks.

2.1 MPC

MPC allows multiple parties to collectively compute on their corresponding inputs without revealing any information about the inputs beyond what can be inferred from the output. One of the most common techniques to realise MPC is secret sharing. Let $x \in \mathbb{G}$ be an element of a finite group. Then we define additive secret share of party i as $\langle x \rangle_i$, such that $x = \sum_i^N \langle x \rangle_i$, where N is the number of computing parties. Thus, to reconstruct the secret from secret shares, the parties send their shares to the party that is supposed to learn the output (can be all parties, or a specific subset), which computes the sum $x = \sum_i^n \langle x \rangle_i$. In the context of PPML, it is common to assume that the clients outsource the computation to a cluster of parties executing an MPC protocol. The most widely used setting involves three computing parties, relying on replicated secret sharing (RSS). In RSS, a secret value is split additively into three random shares, and each party is given two of these shares, such that no single party by itself learns the secret, but any two parties together can reconstruct the secret. To be more precise, we define an RSS share of party i as $\llbracket x \rrbracket_i = [\langle x \rangle_i, \langle x \rangle_{(i+1) \bmod 3}]$, where $x = \sum_{i=1}^3 \langle x \rangle_i$.

2.1.1 Security. In MPC we consider two types of adversaries, namely, *semi-honest* and *malicious*. A semi-honest (also called passive or honest-but-curious) adversary follows the protocol description as defined but tries to infer private information from the protocol transcript. On the other hand, a malicious (or active) adversary is allowed to deviate arbitrarily from the protocol with the intention of gaining information about the private inputs. Note that protocols secure against malicious adversaries are more expensive due to their ability of detecting such malicious behaviour. If the honest parties detect malicious behaviour, they abort the computation.

Moreover, we distinguish security based on the number of corrupted parties. Let N be the number of parties in an MPC protocol. If a protocol is secure in the honest majority setting, it tolerates up to $\lfloor \frac{N-1}{2} \rfloor$ adversary-controlled parties. Conversely, a protocol secure in the dishonest majority setting is able to tolerate up to $N - 1$ corrupted parties.

2.1.2 Fixed-Point Arithmetics. For efficiency reasons, the PPML frameworks work over the ring $\mathbb{Z}_{2^\ell}$ of integers modulo a power of two with the usual choice of $\ell = 32$ or $\ell = 64$. Note that in MPC we require the input to be in a finite domain, however, in machine learning the inputs are real numbers. Hence, we need a way to encode real numbers to elements of a finite ring. This is traditionally implemented with fixed-point arithmetics. Let $\tilde{x} \in \mathbb{R}$. We define $x = \lfloor \tilde{x} \cdot 2^k \rfloor$ to be the *fixed-point representation* of $\tilde{x}$, where k is the fixed-point precision. Now, in order to represent x as an element of $\mathbb{Z}_{2^\ell}$, we define the *encoding* of x to be:

$$z = \begin{cases} x & \text{if } x \geq 0 \\ 2^\ell - |x| & \text{if } x < 0 \end{cases},$$

where $|x| < 2^{\ell-1}$.

In fixed-point arithmetics, the precision is doubled after every multiplication. Hence, to preserve the original precision and avoid an exponential growth in the fractional part, we need to cut the last, say, d bits from the multiplication output. This process is in the

MPC literature normally referred to as truncation. Let x be fixed-point representation of $\tilde{x}$. We can then separate x into the lower d bits representing the fractional part and higher bits representing the integer part of $\tilde{x}$. We write $x = x_1 2^d + x_2$. We then define a truncation of x as $\text{trc}(x) = x_1$.

Truncation is a non-linear operation that brings a high cost in MPC; there has been a substantial effort in the last years to reduce this cost. We distinguish two types of truncation protocols in MPC: *probabilistic* truncation and *faithful* truncation. Probabilistic protocols output the correct truncation result with an error on the least significant bit (LSB) with a probability dependent on the proportion of a ring size to the input size. Next, faithful, or precise truncation protocols always return the correct truncation result. The faithful truncation protocols [8, 9] impose large communication and computational overhead, making them impractical for computationally heavy tasks such as deep learning. On the other hand, it has been shown that probabilistic truncations do not have a negative impact on the accuracy of ML models, despite their probabilistic behaviour. Furthermore, the state-of-the-art probabilistic truncation protocols are much more efficient, requiring at most a single round of communication in the online (input-dependent) phase [16].

2.1.3 SecureML Truncation. In the 2-party computation setting, SecureML [17] is the state-of-the-art probabilistic truncation. In SecureML, the parties hold a 2-party additive sharing of a secret x , such that $\langle x \rangle_1 = x + R$ and $\langle x \rangle_2 = -R$, with R drawn uniformly at random. We will say a 2-party secret sharing *has a randomness* R , if R is the random value forming the secret shares. The SecureML truncation requires the parties to compute $\langle \lfloor x \rfloor \rangle_1 = \text{trc}(\langle x \rangle_1)$ and $\langle \lfloor x \rfloor \rangle_2 = 2^\ell - (2^\ell - \langle x \rangle_2)$. Here, $\langle \lfloor x \rfloor \rangle$ denotes sharing of the SecureML truncation. Note that this truncation does not require any communication between the parties and only comprises cheap local operations.

2.1.4 ABY3-Style Truncation. In the 3-party honest majority setting, ABY3 [16] proposed two approaches to secure truncation: one in the semi-honest setting and the other in the malicious setting. In the semi-honest setting, they make use of replicated secret sharing, where any two parties can convert their secret shares into a 2-party secret sharing. These parties then perform the 2-party semi-honest SecureML truncation and re-share the result obtaining 3-party RSS of the truncation. In malicious setting, they introduce a protocol requiring one round of communication in the online phase, however, with an expensive offline (input-independent) phase. As a result, many subsequent works aimed to optimise the offline phase of the ABY3 truncation [3, 13]. The most recent work on 3-party maliciously secure truncation is MaSTer [24], achieving an amortised cost of a single communication round without the need for any offline (input-independent) phase.

2.2 Neural Networks

Machine learning systems typically involve two distinct phases. First, a training phase, in which a model is trained by learning the relationships between training samples' features and their corresponding labels. Second, an inference phase, during which the trained model predicts the label of an arbitrary sample based on the learned relationships. We refer to the inference phase as the

forward pass of the network. In the case of fully connected layers in a deep neural network, let $\mathbf{x}$ be the queried input, and let $W_i, \mathbf{b}_i$ represent the trained model parameters, consisting of the weight matrix and bias vector for layer L_i , respectively. Let f_i^A denote the corresponding activation function of layer L_i . Then, the forward pass through i -th layer yields the output vector $\mathbf{y}_i$:

$$L_i(\mathbf{x}) : \mathbf{y}_i = f_i^A(\mathbf{x}W_i + \mathbf{b}_i).$$

We then define the inference of an n -layer network as $f_{\text{infer}}(\mathbf{x}) = (L_n \circ \dots \circ L_1)(\mathbf{x})$.

2.2.1 Activation Functions. Activation functions introduce non-linearity into the network, allowing it to learn complex patterns. Common activation functions include:

- ReLU (Rectified Linear Unit): $\text{ReLU}(x_i) = \max(0, x_i)$
- Sigmoid: $\sigma(x_i) = \frac{1}{1+e^{-x_i}}$
- Softmax: $\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^K e^{x_j}}$, where $K = |\mathbf{x}|$.

2.3 Attacks on Machine Learning

In this section we briefly review the existing attacking strategies on black-box ML models, from the perspective of a client sending queries [20].

2.3.1 Adversarial Examples. Adversarial examples are carefully crafted input samples designed to mislead machine learning models. These inputs contain imperceptible perturbations (noise) that cause incorrect predictions with high confidence. Popular methods for generating adversarial examples include the Fast Gradient Sign Method (FGSM) [7] and Projected Gradient Descent (PGD) [23].

2.3.2 Model Inversion. Model inversion attacks [5] aim to reconstruct features of the training data from the output of a trained model. Given a model's prediction or confidence scores for a specific input, an adversary attempts to recover sensitive information about the original data samples. A common approach involves optimizing an input x to maximize the model's confidence on a specific class.

2.3.3 Model Extraction. Model extraction attacks, also known as model stealing attacks, aim to replicate the functionality of a target model by querying it and collecting its outputs. An adversary constructs a surrogate model that approximates the target model by using the predictions obtained from querying the target on various inputs. The attacker can then build a near-identical copy of the model without access to its training data or internal parameters. Canales Martinez et al. [15] have shown that these attacks can practically and accurately extract the complete model.

2.3.4 Membership Inference. Membership inference attacks aim to determine whether a particular data sample was part of the training dataset of a machine learning model. These attacks leverage the fact that models often behave differently on training and non-training samples [21]. The goal is to infer membership based on the output confidence scores or the model's behaviour.

3 MaSTer

The MaSTer [24] protocol is a maliciously secure truncation protocol leveraging repeated execution of an inexpensive passively secure truncation protocol from SecureML [17] coupled with a

consistency check for detecting malicious behaviours. The MaSTer protocol is designed for the 3-party setting with parties holding replicated secret shares of an input $\llbracket z \rrbracket$. A designated pair of the three parties then transforms the RSS to an additive 2-party secret sharing. These two parties then perform local semi-honest SecureML truncation as described in Section 2.1.3. As the authors of MaSTer note, their protocol exhibits probabilistic behaviour inherited from SecureML. As it is crucial for understanding the adversarial capabilities in MaSTer, we reiterate the error analysis of SecureML protocol below.

THEOREM 3.1 (SECUREML TRUNCATION ERROR [17, 24]). *Let $R \in [0, 2^\ell]$, let k be the fixed-point precision and let z be the encoding of x . We decompose $x = x_1 2^k + x_2$ with $0 \leq x_1 < 2^{\ell-k}$, $0 \leq x_2 < 2^k$, and $R = R_1 2^k + R_2$ with $0 \leq R_1 < 2^{\ell-k}$, $0 \leq R_2 < 2^k$. Then, $\text{trc}(z + R) + (2^\ell - \text{trc}(R)) = \text{trc}(x) + c$ with probability $1 - 2^{\ell_{x+1}-\ell}$, where*

$$c = \begin{cases} 1 & \text{if } x \geq 0 \wedge x_2 + R_2 \geq 2^k \\ -1 & \text{if } x < 0 \wedge |x_2| > R_2 \\ 0 & \text{else} \end{cases}.$$

The core idea behind the consistency check in MaSTer involves comparing the results of two independent executions of the SecureML truncation protocol between different pairs of parties. If the protocol were purely deterministic, verifying correctness would be straightforward—simply checking whether the difference is exactly 0. However, as the truncation exhibits probabilistic error, the final check then entails checking whether $\langle \lfloor z \rfloor \rangle - \langle \lfloor z' \rfloor \rangle \in \{-1, 0, 1\}$. This is because $\langle \lfloor z \rfloor \rangle - \langle \lfloor z' \rfloor \rangle = \text{trc}(x) + c - (\text{trc}(x') + c') = c - c'$. Applying Theorem 3.1, we get $c - c' \in \{-1, 0, 1\}$. As a result, an adversary can influence the output of the protocol with an additive error Δ as long as $c - c' + \Delta \in \{-1, 0, 1\}$. We detail the concrete adversarial capabilities in Section 3.1.

3.1 Analysis of Adversarial Influence

The authors of MaSTer [24] provided a detailed analysis of the adversarial capabilities, noting that the adversary can influence the result without getting detected with any $|\Delta| \leq 2$. Note that the probability of detection depends on both the choice of Δ and the magnitude and sign of the truncated value, as depicted in MaSTer. To be more concrete, the authors of MaSTer analysed the case of $\Delta = \pm 2$, concluding that an adversary is caught with a probability at least $\frac{3}{4}$, omitting the case of $\Delta = \pm 1$ as it cannot be lower-bounded. Nevertheless, when addressing strategies for exploiting MaSTer's adversarial model, it is important to understand the adversarial capabilities when $\Delta = \pm 1$.

In Theorem 3.2 we thus extend the analysis from MaSTer [24] for adversarial influence to determine the probability of detection when $\Delta = \pm 1$.

THEOREM 3.2 (DETECTION PROBABILITY). *Let z be an encoding of x . Let $\langle \lfloor z \rfloor \rangle, \langle \lfloor z' \rfloor \rangle$ be 2-party sharings of $\lfloor z \rfloor, \lfloor z' \rfloor$ with randomness R and R' , respectively. Let the sign function be defined as*

$$\text{sgn}(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x = 0 \\ -1 & \text{if } x < 0 \end{cases}.$$

The probability of an adversary influencing the truncation result with a fixed additive error $\Delta = \pm 1$ without getting detected is $1 - \left(\left(\frac{\mathbb{E}[x_2]}{2^k} - \frac{(\mathbb{E}[x_2])^2}{2^{2k}} \right) \cdot (\Pr[\text{sgn}(x) = -1] + \Pr[\text{sgn}(x) = 1]) \right)$, where $\mathbb{E}[x_2]$ denotes the expected value of the truncated part.

PROOF. In order for an adversary to alter the truncation result with an additive error $\Delta = \pm 1$ without getting detected, the consistency check needs to hold: $\langle \lfloor z \rfloor \rangle - \langle \lfloor z' \rfloor \rangle + \Delta \in \{-1, 0, 1\}$. This simplifies to $c - c' + \Delta \in \{-1, 0, 1\} \Rightarrow |c - c' + \Delta| \leq 1$, where c, c' are the truncation errors from $\langle \lfloor z \rfloor \rangle$ and $\langle \lfloor z' \rfloor \rangle$, respectively. Following the notation from MaSTer [24], we thus want to calculate

$$\Pr[|c - c' + \Delta| \leq 1 \mid \text{sgn}(x) \wedge \Delta = \pm 1],$$

Firstly, note that this is equivalent to

$$\begin{aligned} & 1 - \Pr[|c - c' + \Delta| \leq 2 \mid \text{sgn}(x) \wedge \Delta = \pm 1] \\ &= 1 - \Pr[|c - c' + \Delta| = 2 \mid \text{sgn}(x) \wedge \Delta = \pm 1], \end{aligned}$$

as $|c - c'| \leq 1$. Hence, we will examine the probability $\Pr[|c - c' + \Delta| = 2 \mid \text{sgn}(x) \wedge \Delta = \pm 1]$. Following the approach of MaSTer [24] we get:

$$\begin{aligned} & \Pr[|c - c' + \Delta| = 2 \mid \text{sgn}(x) \wedge \Delta = \pm 1] \\ &= \Pr[c = 1 \wedge c' = 0 \mid \text{sgn}(x) = 1 \wedge \Delta = 1] \\ & \quad \cdot \Pr[\text{sgn}(x) = 1] \cdot \Pr[\Delta = 1] \\ & \quad + \Pr[c = 0 \wedge c' = 1 \mid \text{sgn}(x) = 1 \wedge \Delta = -1] \\ & \quad \cdot \Pr[\text{sgn}(x) = 1] \cdot \Pr[\Delta = -1] \\ & \quad + \Pr[c = -1 \wedge c' = 0 \mid \text{sgn}(x) = -1 \wedge \Delta = -1] \\ & \quad \cdot \Pr[\text{sgn}(x) = -1] \cdot \Pr[\Delta = -1] \\ & \quad + \Pr[c = 0 \wedge c' = -1 \mid \text{sgn}(x) = -1 \wedge \Delta = 1] \\ & \quad \cdot \Pr[\text{sgn}(x) = -1] \cdot \Pr[\Delta = 1]. \end{aligned}$$

We assume an adversary with a fixed Δ taking a single value (± 1) with probability 1. Thus we get

$$\begin{aligned} & \Pr[|c - c' + \Delta| = 2 \mid \text{sgn}(x)] \\ &= \Pr[c = 1 \wedge c' = 0 \mid \text{sgn}(x) = 1] \cdot \Pr[\text{sgn}(x) = 1] \\ & \quad + \Pr[c = 0 \wedge c' = 1 \mid \text{sgn}(x) = 1] \cdot \Pr[\text{sgn}(x) = 1] \\ & \quad + \Pr[c = -1 \wedge c' = 0 \mid \text{sgn}(x) = -1] \cdot \Pr[\text{sgn}(x) = -1] \\ & \quad + \Pr[c = 0 \wedge c' = -1 \mid \text{sgn}(x) = -1] \cdot \Pr[\text{sgn}(x) = -1]. \end{aligned}$$

Let us start by analysing the first case of

$$\Pr[c = 1 \wedge c' = 0 \mid \text{sgn}(x) = 1] \cdot \Pr[\text{sgn}(x) = 1].$$

Case $c = 1 \wedge c' = 0$. From Theorem 3.1 we get that $c = 1$ if $x_2 + R_2 \geq 2^k$ and $c' = 0$ when $x_2 + R'_2 < 2^k$ for $x \geq 0$. Therefore,

$$\begin{aligned} & \Pr[c = 1 \wedge c' = 0 \mid \text{sgn}(x) = 1] \cdot \Pr[\text{sgn}(x) = 1] \\ &= \Pr[x_2 + R_2 \geq 2^k \wedge x_2 + R'_2 < 2^k] \cdot \Pr[\text{sgn}(x) = 1]. \end{aligned}$$

Since $x_2, R_2, R'_2 \in [0, 2^k)$ with R, R' uniformly random by definition, we get

$$\begin{aligned} & \Pr[x_2 + R_2 \geq 2^k \wedge x_2 + R'_2 < 2^k] \cdot \Pr[\text{sgn}(x) = 1] \\ &= \frac{\mathbb{E}[x_2]}{2^k} \cdot \left(1 - \frac{\mathbb{E}[x_2]}{2^k} \right) \cdot \Pr[\text{sgn}(x) = 1] \\ &= \left(\frac{\mathbb{E}[x_2]}{2^k} - \frac{(\mathbb{E}[x_2])^2}{2^{2k}} \right) \cdot \Pr[\text{sgn}(x) = 1]. \end{aligned}$$

Note that the subsequent cases follow the same pattern resulting in the same probability, as detailed in MaSTer [24]

Hence, given a fixed $\Delta = \pm 1$, the probability of an adversary passing the consistency check without getting detected by the protocol is:

$$\begin{aligned} & 1 - \Pr[|c - c' + \Delta| \leq 2 \mid \text{sgn}(x) \wedge \Delta = \pm 1] \\ &= 1 - \left(\left(\frac{\mathbb{E}[x_2]}{2^k} - \frac{(\mathbb{E}[x_2])^2}{2^{2k}} \right) \cdot (\Pr[\text{sgn}(x) = -1] + \Pr[\text{sgn}(x) = 1]) \right). \end{aligned}$$

□

4 Adversarial Strategies

In this section we evaluate the different approaches of an adversary trying to exploit the ability of adding $\Delta = \pm 1$ during the computation of the MaSTer [24] truncation, without getting detected. Note that to exploit this property of MaSTer, the attacker must gain control over a computing party. By the usual definition of malicious security in MPC, such an attacker is detected whenever he tries to deviate from the protocol by modifying certain values. However, as the authors of MaSTer note, MaSTer introduces a new adversarial model, in which a malicious attacker is detected with a certain probability < 1 , when modifying the values inside the protocol with an additive error of $\Delta = \pm 1$. The authors of MaSTer further provide an evaluation of this vulnerability by simulating an attacker with no particular strategy, adding $\Delta = \pm 1$ at random. They conclude such attacker cannot influence the classification result, as the impact of the error is negligible for practical values of fixed-point precision.

In this paper, we aim to design a more elaborate strategy for the attacker using realistic and practically deployable strategies and tools at their disposal. We focus on the scenario of machine learning as a service (MLaaS), in which the inference computation is outsourced to three computing parties. Both the input and the machine learning model remain private from the computing parties throughout the computation. The output is revealed to the client requesting the computation, hence the parties remain oblivious to the output as well. However, the model provider and the client are out of the scope of the MPC threat model. As a result, an attacker might collude with a model provider or certain clients.

As a malicious party participating in the MPC protocol, the adversary does not have direct access to the computation's intermediate values due to the inherent privacy guarantees of MPC. The only information an attacker gains by corrupting a party is the knowledge of the computing circuit, in our case, the ML network architecture. Ultimately, the attacker's goal is to misclassify a sample being inferred by modifying the result in each truncation. As noted in MaSTer [24], introducing an additive error of $\Delta = \pm 1$ at random does not yield misclassification. For a more strategic approach, the

attacker requires some level of knowledge about the deployed ML model.

To acquire this knowledge, we assume that the attacker is capable of executing standard model inference attacks from the position of an impersonated client. These attacks may include model inversion, model extraction, or membership inference (cf. Section 2.3), allowing the attacker to reconstruct a model that closely approximates the deployed one. For the remainder of this work we assume an attacker has executed such attack and is in possession of an *approximate* trained model along with a representative dataset. With an approximate model in hand, the attacker can construct attack vectors based on several strategies. We introduce two main approaches: reference- and optimisation-based attacks, each leveraging different techniques to manipulate inference results. In Sections 4.1 and 4.2 we provide a detailed analysis of these two attack strategies.

A brief overview of the attack strategy is as follows. The attacker has the capability to influence every truncation in the computation. Since each multiplication is typically followed by a truncation, the attacker can introduce an additive error of $\Delta = \pm 1$ after every multiplication in the circuit. As previously discussed, the process of inference consists of computing a forward pass of each layer in the network. The output of each layer is computed as $y_i = f_i^A(\mathbf{x}W_i + \mathbf{b}_i)$. The forward pass of a single layer thus involves evaluating a single multiplication $\mathbf{z} = \mathbf{x} \cdot W_i$. Let m and n denote the input and output dimensions of the layer, respectively. Then, $\mathbf{x}^{1 \times m}$ is an m -dimensional vector and $W_i^{m \times n}$ is an $m \times n$ matrix. Their product $\mathbf{z}^{1 \times n}$ is thus an n -dimensional vector, where each element is a sum of m products. Formally, the j -th element of $\mathbf{z}$ is computed as $z_j = \sum_{k=1}^m \mathbf{x}_k \cdot (W_i)_{kj}$. Therefore, to simplify the attack, finding an error of $\Delta = \pm 1$ for each multiplication is equivalent to finding a set of attack vectors $\{\mathbf{a}_i\}$ for each layer L_i , such that each element of $\mathbf{a}_i$ is bounded by $b = \pm m \cdot 2^{-d}$, where m is the input dimension as before, and d is the fixed-point precision used in the MPC protocol.

NOTE 1 (OPTIMISED CIRCUIT). *The authors of MaSTer [24] note that to limit the adversarial influence and to optimise the overall communication and computation overhead, the truncation protocol can be executed on the result of the dot product, rather than for each point-wise product separately. To be more precise, before we assumed in the dot product $\mathbf{z}_j = \sum_{k=1}^m \mathbf{x}_k \cdot (W_i)_{kj}$ the truncation is applied m times on every $\mathbf{x}_k \cdot (W_i)_{kj}$ separately, for all $k \in [1, \dots, m]$. In the optimised version of the computation circuit, the truncation is performed once on $\mathbf{z}_j$. As a result, each element of the attack vector $\mathbf{a}_i$ is then bounded by $b = \pm 2^{-d}$, with d denoting the fixed-point precision.*

4.1 Reference-Based Attack

The attacker’s goal is to induce misclassification of client queries during inference. Note that the attacker does not possess any information about the clients’ input, unless he mimics one of the clients in which case the attacker causes incorrect classification of its own sample. Hence, the attacker needs to obtain an attacking strategy that is oblivious to the input, such that when applied to an arbitrary sample, the result is a misclassification. The attacker thus needs to obtain some general *reference* for misclassification. There are two approaches to this, using a standard inference and using adversarial examples. We detail both below. The two techniques differ in the

underlying approach to classification. While the *inference* attack is more suited for targetted attacks, where adversary picks a target class for misclassification, the approach using adversarial examples is more suited for general misclassification attacks.

4.1.1 Inference-Based Reference. Assume that an attacker targets a specific class for misclassification. As an example consider the MNIST dataset consisting of hand-written digits from 0-9. The attacker aims to misclassify every sample to a certain digit, for example “0”. To achieve this, the attacker first obtains a *reference* output for each layer. This is done by running an inference on a correctly classified sample of the target class—in this case, a digit “0”—and recording the intermediate outputs of each layer throughout the network. Naturally, for any sample passing through the network, if the output of each layer can be modified to resemble the recorded output, the classification will result in class “0”.

More formally, let l_t be a chosen target label. Let $S_t = \{\mathbf{x} \mid f_{\text{Infer}}(\mathbf{x}) = l_t\}$ be the set of all inputs that are classified as the target label. We define the reference matrix as:

$$M_i^{\text{ref}} = \mathbf{x}W_i \text{ for } \forall \mathbf{x} \in S_t,$$

where M_i^{ref} is the i -th row of the reference matrix M^{ref} .

NOTE 2. *There are a variety of options for choosing the right input $\mathbf{x}$ from the set of inputs S_t to obtain the reference matrix. A naive approach would be to pick a sample at random. Other approaches include choosing a sample with the highest confidence in classification of the targetted class. Further possibilities include computing the statistical mean, median or medoid of all samples in S_t . Moreover, notice one can run similar statistical analysis on the output vectors of each layer. This entails computing M^{ref} , $\forall \mathbf{x} \in S$, and then computing statistical mean, median or medoid on the obtained M^{ref} . We will evaluate the above-mentioned techniques empirically. We present the results in Section 5.2.*

To complete the attack, the attacker must compute the appropriate set of attack vectors. This is achieved by leveraging the reference output, ensuring that the modified inference converges toward the desired misclassification. At a high level, the attacker selects samples with labels different from the target class and runs inference on these samples. By observing the intermediate outputs of each layer and comparing them to the recorded reference outputs, the attacker determines the discrepancy at each layer. This difference serves as the attack vector, which, when applied, gradually shifts the network’s activations toward the reference, ultimately leading to misclassification. More formally, let $S_{\bar{t}} = \{\mathbf{x} \mid f_{\text{Infer}}(\mathbf{x}) \neq l_t\}$ be the set of samples that are not classified as the target label. Let $\mathbf{x} \in S_{\bar{t}}$. Then the attack vector for layer L_i can be computed as

$$\mathbf{a}_i = \text{clip} \left(M_i^{\text{ref}} - \mathbf{x}W_i, -b, b \right), \quad (1)$$

where $\text{clip}(\cdot, -b, b)$ denotes the clipping function bounding all values within the interval $[-b, b]$.

The attacker then runs the attack by applying $\Delta = \pm 1$ according to the attack vector. Thus, for each layer L_i , the output becomes

$$y_i = f_i^A(\mathbf{x}W_i + \mathbf{a}_i + \mathbf{b}_i) \quad (2)$$

$$= f_i^A \left(\mathbf{x}W_i + \text{clip} \left(M_i^{\text{ref}} - \mathbf{x}W_i, -b, b \right) + \mathbf{b}_i \right). \quad (3)$$

Note that the higher the bound the outputs gets closer to the reference M_i^{ref} and thus the misclassification to the target class l_t . Further, higher bound is equivalent to lower fixed-point precision, hence we can expect the effectiveness of the attack to be lower with rising fixed-point precision. Note that as the bound increases, the output values shift closer to the reference M_i^{ref} , thereby increasing the likelihood of misclassification into the target class l_t .

4.1.2 Adversarial Examples-Based Reference. Another approach for obtaining a reference output is using adversarial examples (cf. Section 2.3.1). In this scenario the attacker does not aim to misclassify a sample towards a concrete target class, but rather achieve a misclassification of any kind. For instance, consider a classification model for arrhythmia detection on heartbeat data. The attacker’s goal might be to misclassify samples with no arrhythmia as a type of arrhythmia with no particular focus on the concrete type of arrhythmia. Hence, in this case, the attacker targets samples from a concrete target class l_t and attempts to misclassify them as any label $l \neq l_t$. Notice the difference to the previous attack using inference. Previously, the attacker targetted all samples with the goal of misclassification to a concrete target class. In this attack the attacker targets a certain target class of samples with the goal of misclassifying them to any class.

The attack begins with the attacker computing adversarial examples on samples in the dataset S . We implement adversarial examples generation using the cleverhans [20] library in Python. We will denote the adversarial example as $\mathbf{x}^{\text{adv}} = \mathbf{x} + \delta$, where δ represents the small perturbation for the sample. We will then denote the set of adversarial inputs S^{adv} .

Similarly to the approach described above, the attacker computes the reference matrix for correct approach to misclassification. To compute the reference matrix, the attacker creates a set of all adversarial examples, such that the original samples evaluate to the chosen target class. Formally, the adversary picks $\mathbf{x}^{\text{adv}} \in \{\mathbf{x}^{\text{adv}} \mid f_{\text{infer}}(\mathbf{x}) = l_t\}$. The attacker then proceeds to compute the reference matrix in a similar fashion as before: $M_i^{\text{ref}} = \mathbf{x}^{\text{adv}} W_i$, with M_i^{ref} denoting the i -th row of the reference matrix M^{ref} .

As before, there are various approaches for choosing the right $\mathbf{x}^{\text{adv}}$ for computation of the reference matrix. The options follow from Note 2. We evaluate these approaches in Section 5.2.

To compute the set of attack vectors and complete the attack the adversary follows the procedure outlined in Equations 1 and 2.

4.2 Optimisation-Based Attack

So far, we have explored two adversarial strategies, both relying on a reference to construct an attack matrix that induces misclassification. Ultimately, the goal of the attacker is to find the set of attack vectors $\{\mathbf{a}_i\}$ to misclassify a sample towards some target label l_t . Consider an N -layer neural network. The output of the inference of $\mathbf{x} \in \{\mathbf{x} \mid f_{\text{infer}}(\mathbf{x}) = l\}$, with attack vectors, is given by

$$\mathbf{y} = f_N^A \left(W_N \left(\cdots \left(f_1^A (W_1 \mathbf{x} + \mathbf{a}_1 + \mathbf{b}_1) + \mathbf{a}_2 + \mathbf{b}_2 \right) \cdots \right) + \mathbf{a}_n + \mathbf{b}_n \right),$$

with $\arg \max(\mathbf{y})$ corresponding to the predicted label. The goal of the attacker thus, is to find set of attack vectors $\{\mathbf{a}_i\}$, such that $\arg \max(\mathbf{y}) = l_t \neq l$.

To ensure that class l_t is predicted with the highest confidence, we can define the loss function as the negative of the target logit:

$$\mathcal{L}(\{\mathbf{a}_i\}) = -\mathbf{y}_{l_t} \quad \forall i,$$

where $\mathbf{y}_{l_t}$ denotes the logit of the confidence vector $\mathbf{y}$ corresponding to the class l_t . Minimizing $\mathcal{L}$ is equivalent to maximizing $\mathbf{y}_{l_t}$.

For binary classification (with a single output logit and sigmoid activation σ), the loss is defined as:

$$\mathcal{L}(\{\mathbf{a}_i\}) = \begin{cases} -\sigma(y), & \text{if } l_t = 1, \\ \sigma(y), & \text{if } l_t = 0. \end{cases}$$

Furthermore, we require the values of each $\mathbf{a}_i$ to lie within an interval $[-b, b]$, with some bound b .

Thus, the overall constrained optimisation problem is:

$$\min_{\{\mathbf{a}_i\}} \mathcal{L}(\{\mathbf{a}_i\}), \quad \text{subject to } \mathbf{a}_i \in [-b, b], \quad \forall i.$$

The attacker can thus solve the above-described optimisation problem obtaining attack vectors $\{\mathbf{a}_i\}$. In practice, we realise this optimisation problem using a gradient-based optimizer such as Adam [12], which automatically computes the gradients via Back-propagation and adapts the step sizes during training. This method efficiently handles the non-convex nature of the deep network’s loss landscape and ensures that the offsets remain within their prescribed bounds by applying a clipping operation after each update.

5 Evaluation

In this section we present experimental results of our proposed attacks. We start by describing our setup for ML datasets and the corresponding network architectures.

5.1 Models and datasets

We evaluate our attack strategies on a variety of datasets and network architectures. We opt to evaluate the attack on datasets covering multiple datatypes for wider understanding and impact of our attack. In terms of image processing, we choose the widely used MNIST [4] and CIFAR-10 [14] datasets. For time-series data, similar to MaSTer [24], we implement networks on heartbeat data for arrhythmia detection based on the MIT-BIH [18] and PTB [2] datasets available at [6]. Furthermore, we use a Voice recognition dataset [1] containing raw audio samples pre-processed into numerical data. Finally, for categorical dataset, we use the Obesity prediction dataset [19]. In the following we describe network architectures used for evaluation for each dataset. We opted for dense neural network architectures consisting of fully connected layers. We note that in MPC, convolutions are represented by matrix multiplications using Toeplitz transform [?]. Our attacks hence remains directly applicable to convolutional layers, nevertheless, we leave the exploration of a concrete impact as a future work.

5.1.1 MNIST. The MNIST dataset is a well-known benchmark in machine learning and computer vision. It consists of grayscale images of handwritten digits (0-9), each with a resolution of 28×28 pixels. We employ four different dense neural network (DNN) architectures, with 3, 5, 7 and 9 fully connected layers, respectively, with ReLU as the activation in the hidden layers and Softmax in the output layer.

5.1.2 CIFAR-10. The CIFAR-10 dataset comprises colour images with a resolution of 32×32 pixels, divided equally into 10 classes. While convolutional neural networks turned out to be highly applicable for image-like datasets, in this work we focus on fully connected layers. Hence, we evaluate CIFAR-10 on two DNNs with 3 and 5 fully connected layers respectively, with ReLU activation in the hidden layers and Softmax in the output layer. Both of the architectures further comprise the flattening layer at the beginning.

5.1.3 ECG. The Electrocardiogram (ECG) heartbeat classification dataset [10] combines the MIT-BIH arrhythmia [18] and PTB [2] datasets. It comprises pre-processed heartbeat signals with annotations for different types of arrhythmia. Similarly as before, we use two different DNNs with 3 and 5 fully connected layers respectively, with ReLU activation function in the hidden layers and Softmax in the output layer.

5.1.4 VOICE. The Voice Gender recognition dataset [1] consists of acoustic features extracted from voice samples. The dataset contains 3,168 samples from male and female speakers with features representing various acoustic properties. For this we use a 5-layer dense network with 256, 128, 128, 64, and 1 unit, employing ReLU activations in hidden layers and a sigmoid activation in the output for binary classification.

5.1.5 OBESITY. The Obesity Level dataset [19] comprises demographic and lifestyle attributes collected from over 2000 individuals. The objective is a 7-class classification problem, predicting obesity levels from underweight to obesity types. For this classification, we use a 5-layer dense network with 128, 64, 64, 32, and 7 units. ReLU activation is applied to hidden layers, and Softmax activation for the output.

5.2 Attack evaluation

In Section 4 we describe two types of implementations of truncation: An un-optimised version in which the truncation is called after every multiplication in the circuit, and optimised version (cf. Note 1) in which the truncation is only called after the computation of the dot product. The current implementation of MaSTer in MP-SPDZ [11] library includes the former, un-optimised version of truncation implementation. In our work, we implement an attack on both of these. Due to space constraints, however, we present the results for the un-optimised version only.

Note that the strategies for computing the attack vectors outlined in Section 4.1 result in a separate set of attack vectors for each input from the input domain. As a result, the attacker would need to possess some information about the input, in order to apply the corresponding set of attack vectors, which contradicts our assumptions on the capabilities of the attacker. Therefore, to mimic adversarial capabilities, we obtain a single set of attack vectors by averaging them over the samples. This results in a single set of attack vectors that the attacker applies obliviously to the input.

Our evaluation of the attack consists of measuring the success rate (percentage of successfully misclassified samples) against various fixed-point precisions, ranging from 8 to 16. We find this range to be the most realistic regarding implementation in MPC circuits. Lower precisions would naturally degrade the accuracy of the model

without any adversarial influence and so are by default undesirable to use. Higher precisions, on the other hand, would require larger space for inputs resulting in the need for larger computation domain in MPC (larger ring size) which in turn slows down computation due to higher communication overhead. We find that most of the frameworks implementing machine learning in MPC use fixed-point precision within the highlighted range, with state-of-the-art in 3-party computation using RSS, Falcon [22], operating on 13 bits of precision.

In order to proceed with evaluation of the attacks, we need to establish the best method for obtaining the reference matrix M^{ref} . As described in Note 2, there are various methods for obtaining such a matrix. We compare the techniques from Section 4 on different datasets, using the corresponding 5-layer network as a representative. In Figures 1a and 1b we depict the success rates for the inference-based reference attack and adversarial examples-based reference attack, respectively, on the MNIST dataset, as an example. The figures for other datasets are presented in Appendix A.

Note that the task of adversarial examples is to cause misclassification by adding small noise imperceptible to the human eye. It is reasonable to apply such attacks on image datasets (MNIST and CIFAR10). We find that the attack works reasonably well on the ECG heartbeat dataset as well. However, it is counter-intuitive (and does not provide meaningful results) to try to create adversarial examples for tabular data (VOICE and OBESITY datasets). Hence, we omit experiments on these using adversarial examples.

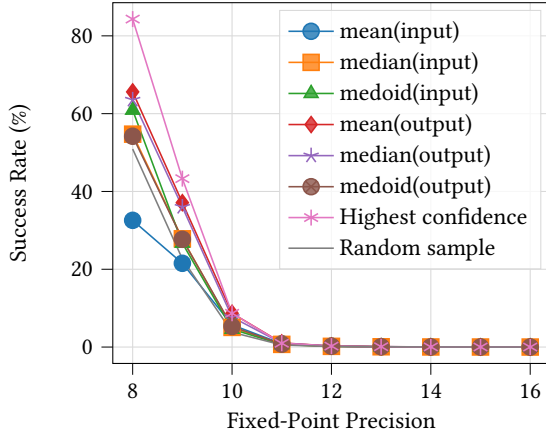
From the analysis of different techniques we observe that different aggregation methods perform differently on various datasets. We choose the best performing aggregation technique for each dataset and continue our experiments with the technique only. We present the chosen technique for each configuration in Table 1.

Table 1: Overview of chosen aggregation techniques for obtaining reference matrix.

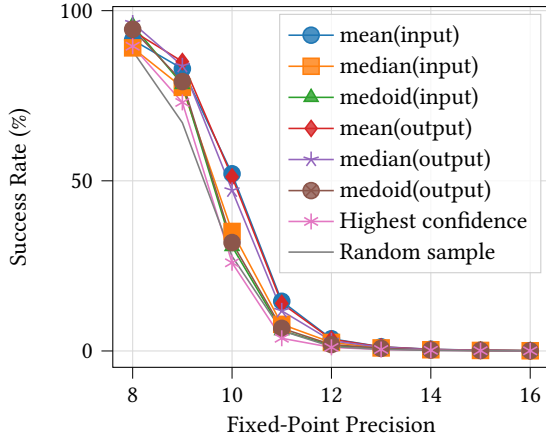
Dataset	Attack	Aggregation technique
MNIST	Inference	Highest confidence
	Adversarial examples	mean(output)
CIFAR10	Inference	mean(output)
	Adversarial examples	medoid(input)
ECG	Inference	mean(output)
	Adversarial examples	mean(output)
VOICE	Inference	medoid(output)
OBESITY	Inference	median(input)

Having chosen the aggregation techniques for obtaining the reference matrix, we proceed with evaluation of the attack on the chosen networks for each dataset. In this work we present results for 5-layer networks only, we refer the reader to our implementation¹ for extended results on different networks. We evaluate the attack by calculating the success rate for various target labels. Recall that, in the context of inference and optimisation attacks, the attacker aims to misclassify all incoming samples to a concrete target label. In the context of adversarial examples attack, the attacker targets a

¹<https://github.com/KULeuven-COSIC/MaSTer-Attack>



(a) Averaged success rate over labels for MNIST using the *inference* attack.



(b) Averaged success rate over labels for MNIST using the *adversarial examples* attack.

Figure 1: Evaluation of techniques for obtaining reference matrix.

class of samples with a concrete target label with the intention of misclassifying them.

Firstly, we evaluate the effectiveness of each attack separately on different datasets and compare the success rates of the attack for concrete labels. The results for the inference-based reference attack are presented in Appendix B, for the adversarial examples-based reference attack in Appendix C and for optimisation-based attack in Appendix D.

We make the following observations.

- We observe that the success rate varies across different labels. This suggests certain labelled samples differ significantly from the others and are thus more difficult to misclassify. As an example of this behaviour consider the MNIST dataset. We find the attacks on label 1, with the goal of misclassifying to label 1 perform significantly worse than for other labels. On the other hand, the attack with adversarial examples performs noticeably well on label 1.

Note that the MNIST dataset consists of greyscale images of hand-written numbers. The number 1 is visually distinct from other numbers, requiring far fewer pixels to be "turned on." In contrast, most other digits have more complex shapes with a higher number of active pixels. As a result, we conclude that for the MNIST dataset, the attacks find it difficult to turn off pixels, but easy to turn on more pixels to misclassify to other numbers.

- We observe that the success rate is higher for more complicated networks with more neurons. In particular, the success rate is higher for networks on image datasets (MNIST and CIFAR10) than those on tabular data such as the OBE-SITY dataset. We further support our claim by comparing success rates for different networks over a single dataset. In Figure 2, we depict a comparison of different networks on the MNIST dataset, with the networks consisting of more neurons (hence more multiplications) achieving higher success rates.
- Regarding the performance of attack strategies, we observe that the adversarial examples-based reference attack performs marginally better than the others. To be more concrete, the attack using adversarial examples achieves a success rate of misclassification about 20% of the time for 13 bits of precision on the CIFAR10 dataset. The success rate of inference-based and optimisation-based attacks approaches 0% for the same configuration. However, note that the attack using adversarial examples is not a targeted attack, i.e. the goal is a misclassification to any class. For targeted attacks, we compare the inference and optimisation attacks directly. We observe a higher success rate of the optimisation attack across all datasets. We highlight the comparison between the two attack methods across different datasets in Figure 3.
- Finally, as expected, we observe a natural trend of dropping the success rate of the attacks with raising fixed-point precision. Specifically, for 8 bits of precision the attacks achieve almost 100% of accuracy on all but the OBESITY dataset. On the other hand, for 16 bits of precision all attacks succeed with a rate close to, if not 0%.

5.3 Adversarial capabilities

In the previous section we analysed the effectiveness of our attacks on ML networks with no restriction on adversarial capabilities in real-world deployment. In particular, we showcase that the attack succeeds with a reasonable success rate for lower values of fixed-point precision, assuming an adversary can add $\Delta = \pm 1$ in each iteration of truncation. In our case above, this is equivalent to adding $\Delta = \pm 1$ after each multiplication in the circuit. Note that in Section 3.1 we analysed the behaviour of the MaSTer consistency check when the adversary attempts to add an influence of ± 1 . In particular, Theorem 3.2 states the adversary succeeds in cheating without violating the consistency check and causing the MPC parties to abort with probability

$$1 - \left(\left(\frac{\mathbb{E}[x_2]}{2^k} - \frac{(\mathbb{E}[x_2])^2}{2^{2k}} \right) \cdot (\Pr[\text{sgn}(x) = -1] + \Pr[\text{sgn}(x) = 1]) \right).$$

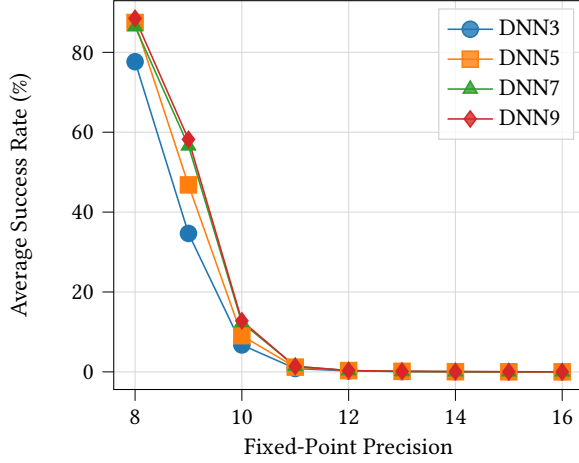


Figure 2: Comparison of success rates across different networks over MNIST dataset. DNN refers to a dense neural network followed by a number of fully connected layers.

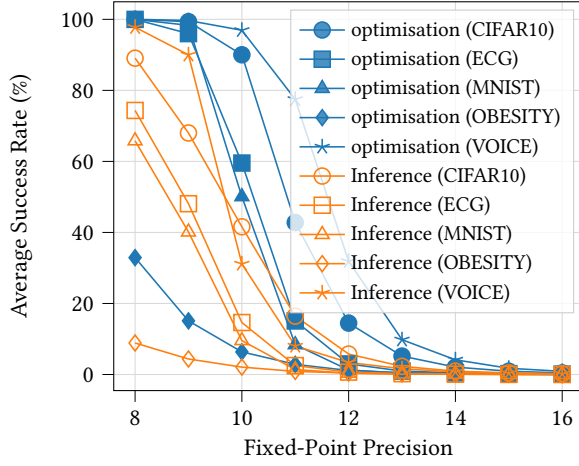


Figure 3: Comparison of success rates across different attacks (inference-based and optimisation-based) on all datasets.

In this section we aim to evaluate the precise probability the attacker will succeed in cheating in each iteration of the truncation. We perform statistical analysis on the values being truncated throughout each network. We measure the distribution of the truncated values, calculating the expected value $\mathbb{E}[x_2]$, where x_2 denotes the truncated part of input x . We further calculate the distribution of $\text{sgn}(x)$. Based on the obtained statistics, we are able to calculate the probability of an adversary cheating without getting detected. We observe that the truncated values closely resemble uniform distribution, and in fact, result in an expected value of $\mathbb{E}[x_2] \approx 2^{d-1}$, where x_2 is the truncated value and d denotes precision. We present the results for precision 16 in Table 2. Indeed, $\mathbb{E}[x_2] \approx 2^{15}$. We conclude that the adversary succeeds in cheating with probability $\frac{3}{4}$ for every instantiation of MaSTer truncation. For a statistical security parameter $\sigma = 40$ suggested by authors of MaSTer, the attacker can then

influence only up to $\log_{\frac{3}{4}} 2^{-40} = 96$ truncations. In Section 5.3.1, we thus evaluate the success rate for a real-world attacker.

Table 2: Analysis of truncated values in a network determining the probability of adversary cheating without getting detected $\Pr[X]$. Here x_2 denotes the part of the input being truncated and $X = 1 - \Pr[|c - c' + \Delta| \leq 2 \mid \text{sgn}(x) \wedge \Delta = \pm 1]$.

Dataset	Network	$\text{sgn}(x)$ (%)			$\mathbb{E}[x_2]$	$\Pr[X]$
		1	0	-1		
MNIST	DNN3	51	0	49	32746	$\approx \frac{3}{4}$
	DNN5	33	0	67	32775	
	DNN7	23	0	77	32766	
CIFAR-10	DNN3	11	0	89	32760	$\approx \frac{3}{4}$
	DNN5	14	0	86	32767	
ECG	DNN3	54	0	46	32761	$\approx \frac{3}{4}$
	DNN5	57	0	43	32774	
VOICE	DNN5	48	0	52	32772	
OBESITY	DNN5	46	0	54	32717	

5.3.1 Budget attack. Here, we introduce a "budget" attack, where the adversary operates under a limited manipulation budget to avoid detection. In this attack, after obtaining the set of attack vectors, the attacker needs to choose strategically which truncations to influence for maximum impact on the network. Note that all hidden layers in our networks are followed by the ReLU activation function. We implement a strategy for the attacker to prioritise affecting positive values, or values with a possibility of flipping a sign, in order to maximise the impact after the activation.

From the previous sections we observe that the optimisation attack performs the best across different datasets, with 5-layer DNN on CIFAR10 being the most vulnerable. Therefore, we choose to present an evaluation of the budget attack on this network. We demonstrate the impact of this adversary in Figure 4. We observe that for 13 bits of precision, the adversary succeeds less than 1% of the time with probability $\approx 1 - 2^{-40}$ of getting detected.

6 Conclusion

In this work we proposed three adversarial strategies for exploiting vulnerability of MaSTer [24] truncation. We evaluated our attacks on various datasets reporting on success rates for several configurations. We note that the attack heavily relies on the assumption that the adversary can execute a model extraction attack, obtaining a model closely resembling the deployed one being attacked. The model inversion attack, however, requires a large number of queries for a successful execution and can be mitigated by restricting each user to a limited number of queries. On the other hand, such mitigation can be bypassed by impersonating multiple users.

We presented two types of attack scenarios. In the first scenario, we considered an adversary that can influence every truncation in the circuit. In this version of the attack, we observed reasonably high success rates for lower precisions. In the more realistic scenario, however, the attacker's influence is very limited. We presented a budget attack in which we considered the probabilistic success rate of the adversarial influence. We showed the feasibility of such an

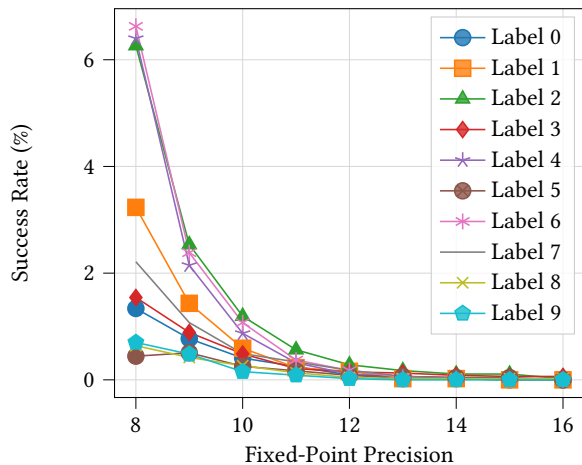


Figure 4: Evaluation of the optimised attack on CIFAR10 dataset with realistic adversarial budget.

attack on the CIFAR10 dataset, achieving distinctively lower success rates, with an exceptionally high probability of protocol aborts.

We have, therefore, demonstrated that even though the MaSTer protocol introduces a new adversarial model allowing the adversary to add a certain error, in practice, it is difficult to exploit using a real-world adversary. Nevertheless, for a more powerful adversary that possess some knowledge about the inputs, these attacks become an effective tool for misclassifying samples, if the fixed-point precision is low.

Acknowledgments

The authors would like to acknowledge master’s student Zeyu Guan for his early involvement and thesis work, which contributed to initial discussions on the topic. This work was supported by the Flemish Government through FWO SBO project MOZAIK S003321N and by CyberSecurity Research Flanders with reference number VR20192203. This work was funded in part by the European Network of Excellence dAIEDGE under Grant Agreement Nr. 10112072.

References

- [1] Kory Becker. 2016. Identifying the gender of a voice using machine learning. May2021.[Online]. Available: <http://www.primaryobjects.com/2016/06/22/identifying-the-gender-of-a-voice-using-machine-learning> (2016).
- [2] R. Bousselot, D. Kreisler, and A. Schnabel. 1995. Nutzung der EKG-Signaldatenbank CARDIODAT der PTB über das Internet. *Biomedical Engineering / Biomedizinische Technik* 40, s1 (1995), 317–318.
- [3] Harsh Chaudhari, Rahul Rachuri, and Ajith Suresh. 2020. Trident: Efficient 4PC Framework for Privacy Preserving Machine Learning. In *27th NDSS 2020, San Diego, California, USA, February 23-26, 2020*. The Internet Society. <https://www.ndss-symposium.org/ndss-paper/trident-efficient-4pc-framework-for-privacy-preserving-machine-learning/>
- [4] Li Deng. 2012. The mnist database of handwritten digit images for machine learning research. *IEEE Signal Processing Magazine* 29, 6 (2012), 141–142.
- [5] Hao Fang, Yixiang Qiu, Hongyao Yu, Wenbo Yu, Jiawei Kong, Baoli Chong, Bin Chen, Xuan Wang, and Shu-Tao Xia. 2024. Privacy Leakage on DNNs: A Survey of Model Inversion Attacks and Defenses. *CoRR* abs/2402.04013 (2024). [arXiv:2402.04013](https://arxiv.org/abs/2402.04013) <https://doi.org/10.48550/arXiv.2402.04013>
- [6] A L Goldberger, L A Amaral, L Glass, J M Hausdorff, P C Ivanov, R G Mark, J E Mietus, G B Moody, C K Peng, and H E Stanley. 2000. PhysioBank, PhysioToolkit, and PhysioNet: Components of a New Research Resource for Complex Physiologic Signals. *Circulation* 101, 23 (June 2000), E215–20.
- [7] Ian J Goodfellow, Jonathon Shlens, and Christian Szegedy. 2014. Explaining and harnessing adversarial examples. *arXiv preprint arXiv:1412.6572* (2014).
- [8] Kanav Gupta, Neha Jawalkar, Ananta Mukherjee, Nishanth Chandran, Divya Gupta, Ashish Panwar, and Rahul Sharma. 2023. SIGMA: Secure GPT Inference with Function Secret Sharing. *IACR Cryptol. ePrint Arch.* (2023), 1269. <https://eprint.iacr.org/2023/1269>
- [9] Kanav Gupta, Deepak Kumaraswamy, Nishanth Chandran, and Divya Gupta. 2022. LLAMA: A Low Latency Math Library for Secure Inference. *Proc. Priv. Enhancing Technol.* 2022, 4 (2022), 274–294. <https://doi.org/10.56553/popets-2022-0109>
- [10] Mohammad Kachuee, Shayan Fazeli, and Majid Sarrafzadeh. 2018. ECG Heartbeat Classification: A Deep Transferable Representation. In *2018 IEEE International Conference on Healthcare Informatics (ICHI)*. 443–444. <https://doi.org/10.1109/ICHI.2018.00092>
- [11] Marcel Keller. 2020. MP-SPDZ: A Versatile Framework for Multi-Party Computation. In *CCS ’20: 2020 ACM SIGSAC, Virtual Event, USA, November 9-13, 2020*, Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna (Eds.). ACM, 1575–1590. <https://doi.org/10.1145/3372297.3417872>
- [12] Diederik P Kingma and Jimmy Ba. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* (2014).
- [13] Nishat Koti, Mahak Pancholi, Arpita Patra, and Ajith Suresh. 2021. SWIFT: Superfast and Robust Privacy-Preserving Machine Learning. In *30th USENIX Security 2021, August 11-13, 2021*, Michael Bailey and Rachel Greenstadt (Eds.). USENIX Association, 2651–2668. <https://www.usenix.org/conference/usenixsecurity21/presentation/koti>
- [14] Alex Krizhevsky, Geoffrey Hinton, et al. 2009. Learning multiple layers of features from tiny images. (2009).
- [15] Isaac Andrés Canales Martínez, Jorge Chávez-Saab, Anna Hambitzer, Francisco Rodríguez-Henríquez, Nitin Satpute, and Adi Shamir. 2024. Polynomial Time Cryptanalytic Extraction of Neural Network Models. In *EUROCRYPT 2024 - 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zurich, Switzerland, May 26-30, 2024, Proceedings, Part III (Lecture Notes in Computer Science, Vol. 14653)*, Marc Joye and Gregor Leander (Eds.). Springer, 3–33. https://doi.org/10.1007/978-3-031-58734-4_1
- [16] Payman Mohassel and Peter Rindal. 2018. ABY³: A Mixed Protocol Framework for Machine Learning. In *Proceedings of the 2018 ACM SIGSAC CCS 2018, Toronto, ON, Canada, October 15-19, 2018*, David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang (Eds.). ACM, 35–52. <https://doi.org/10.1145/3243734.3243760>
- [17] Payman Mohassel and Yupeng Zhang. 2017. SecureML: A System for Scalable Privacy-Preserving Machine Learning. In *2017 IEEE SP 2017, San Jose, CA, USA, May 22-26, 2017*. IEEE Computer Society, 19–38. <https://doi.org/10.1109/SP.2017.12>
- [18] George B Moody and Roger G Mark. 2001. The Impact of the MIT-BIH Arrhythmia Database. *IEEE engineering in medicine and biology magazine* 20, 3 (2001), 45–50.
- [19] Fabio Mendoza Palechor and Alexis De la Hoz Manotas. 2019. Dataset for estimation of obesity levels based on eating habits and physical condition in individuals from Colombia, Peru and Mexico. *Data in brief* 25 (2019), 104344.
- [20] Nicolas Papernot, Fartash Faghri, Nicholas Carlini, Ian Goodfellow, Reuben Feinman, Alexey Kurakin, Cihang Xie, Yash Sharma, Tom Brown, Aurko Roy, Alexander Matyasko, Wahid Behzadan, Karen Hambardzumyan, Zhishuai Zhang, Yi-Lin Juang, Zhi Li, Ryan Sheatsley, Abhibhav Garg, Jonathan Uesato, Willi Gierke, Yinpeng Dong, David Berthelot, Paul Hendricks, Jonas Rauber, and Rujun Long. 2018. Technical Report on the CleverHans v2.1.0 Adversarial Examples Library. *arXiv preprint arXiv:1610.00768* (2018).
- [21] Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. 2017. Membership Inference Attacks Against Machine Learning Models. In *2017 IEEE SP 2017, San Jose, CA, USA, May 22-26, 2017*. IEEE Computer Society, 3–18. <https://doi.org/10.1109/SP.2017.41>
- [22] Sameer Wagh, Shruti Tople, Fabrice Benhamouda, Eyal Kushilevitz, Prateek Mittal, and Tal Rabin. 2021. Falcon: Honest-Majority Maliciously Secure Framework for Private Deep Learning. *Proc. Priv. Enhancing Technol.* 2021, 1 (2021), 188–208. <https://doi.org/10.2478/popets-2021-0011>
- [23] Yulong Yang, Chenhao Lin, Xiang Ji, Qiwei Tian, Qian Li, Hongshan Yang, Zhibo Wang, and Chao Shen. 2023. Towards Deep Learning Models Resistant to Transfer-based Adversarial Attacks via Data-centric Robust Learning. *CoRR* abs/2310.09891 (2023). [arXiv:2310.09891](https://arxiv.org/abs/2310.09891) <https://doi.org/10.48550/arXiv.2310.09891>
- [24] Martin Zbudila, Erik Pohle, Aysajan Abidin, and Bart Preneel. 2024. MaSTer: Maliciously Secure Truncation for Replicated Secret Sharing Without Pre-processing. In *CANS 2024, Cambridge, UK, September 24-27, 2024, Proceedings, Part I (Lecture Notes in Computer Science, Vol. 14905)*, Markulf Kohlweiss, Roberto Di Pietro, and Alastair R. Beresford (Eds.). Springer, 49–73. https://doi.org/10.1007/978-981-97-8013-6_3

A Techniques for reference matrix

We present the evaluation of different techniques for aggregation of data to obtain the reference matrix in Figures 5 and 6 for *inference* attack and *adversarial examples* attack, respectively.

B Evaluation of the inference attack

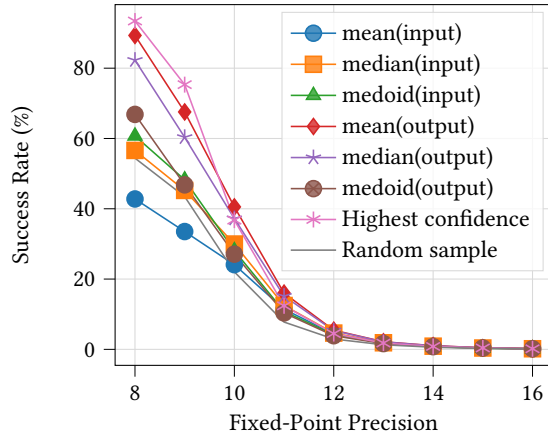
In Figure 7 we present the results of the execution of the inference attack on different datasets.

C Evaluation of the adversarial examples attack

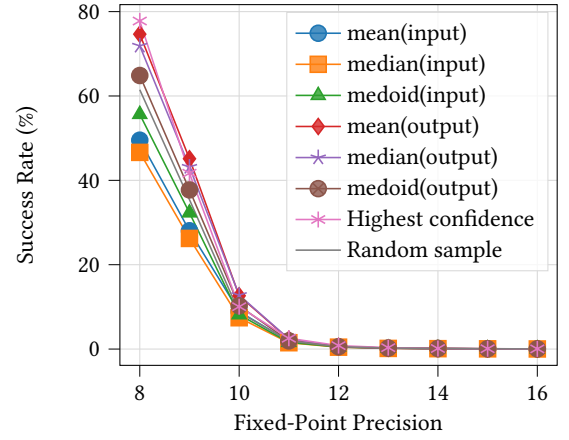
In Figure 8 we present the results of the execution of the adversarial examples attack on different datasets.

D Evaluation of the optimisation attack

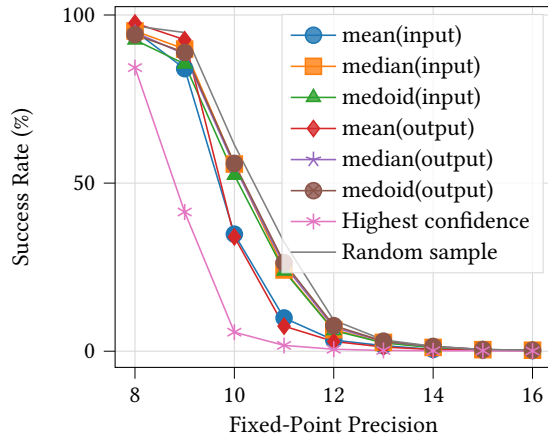
In Figure 9 we present the results of the execution of the optimisation attack on different datasets.



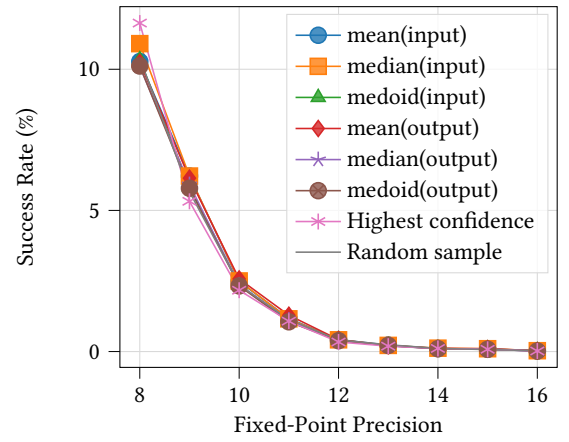
(a) Averaged success rate over labels for CIFAR10 dataset.



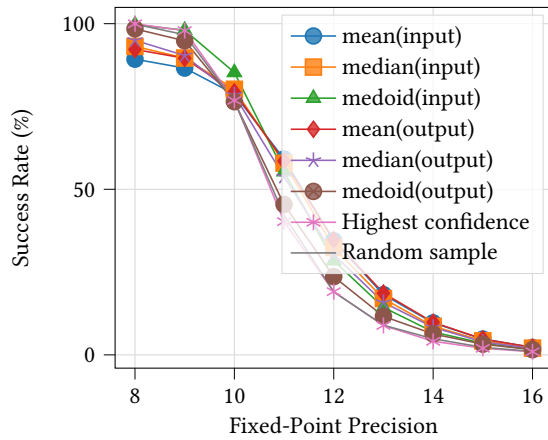
(b) Averaged success rate over labels for ECG dataset.



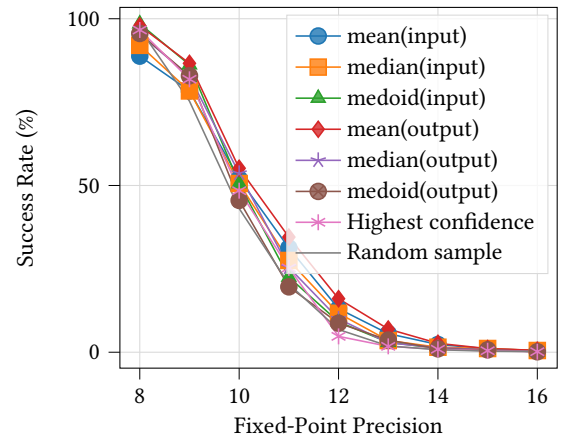
(c) Averaged success rate over labels for VOICE dataset.



(d) Averaged success rate over labels for OBESITY dataset.

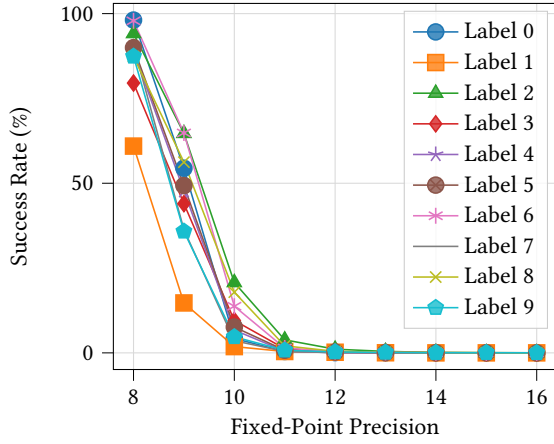
Figure 5: Overview of techniques for obtaining reference matrix using the *inference* attack.

(a) Averaged success rate over labels for CIFAR10 dataset.

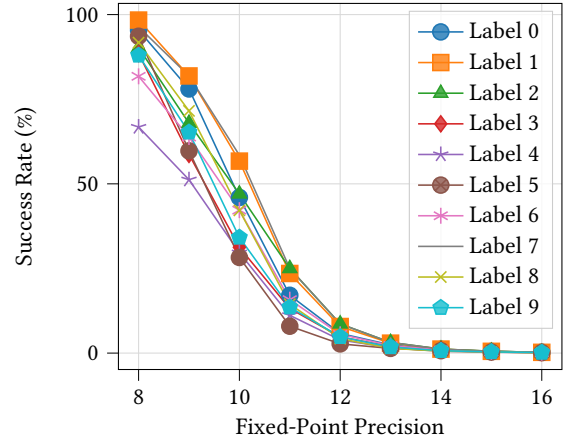


(b) Averaged success rate over labels for ECG dataset.

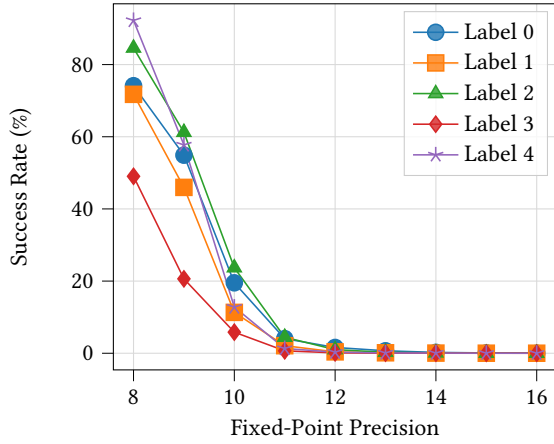
Figure 6: Overview of techniques for obtaining reference matrix using the *adversarial examples* attack.



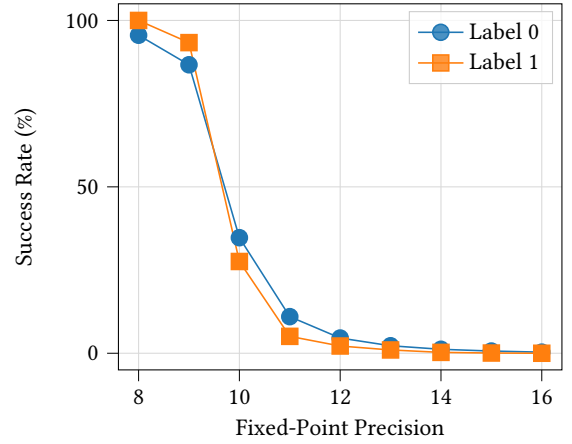
(a) Success rates for misclassifying to a target label on a 5-layer DNN on the MNIST dataset.



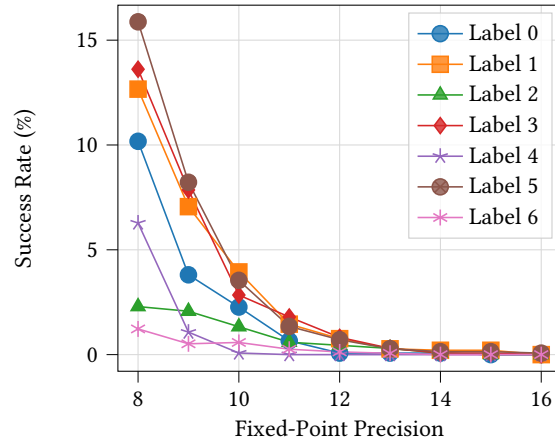
(b) Success rates for misclassifying to a target label on a 5-layer DNN on the CIFAR10 dataset.



(c) Success rates for misclassifying to a target label on a 5-layer DNN on the ECG dataset.

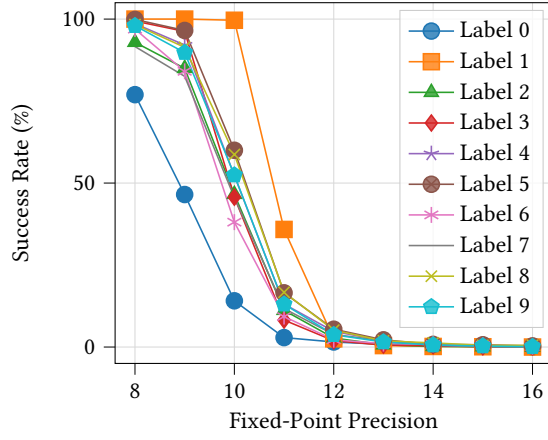


(d) Success rates for misclassifying to a target label on a 5-layer DNN on the VOICE dataset.

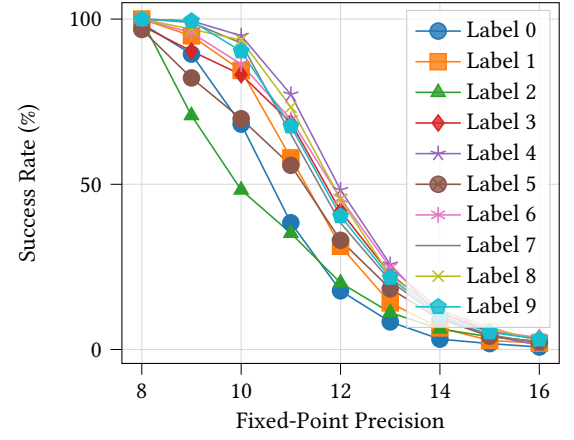


(e) Success rates for misclassifying to a target label on a 5-layer DNN on the OBESITY dataset.

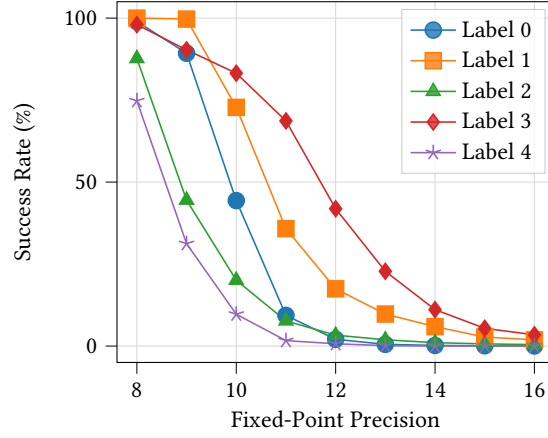
Figure 7: Evaluation of the inference attack on different datasets.



(a) Success rates for misclassifying a target label on a 5-layer DNN on the MNIST dataset.

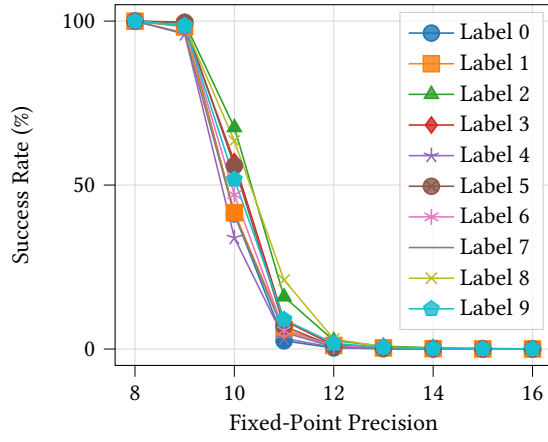


(b) Success rates for misclassifying a target label on a 5-layer DNN on the CIFAR10 dataset.

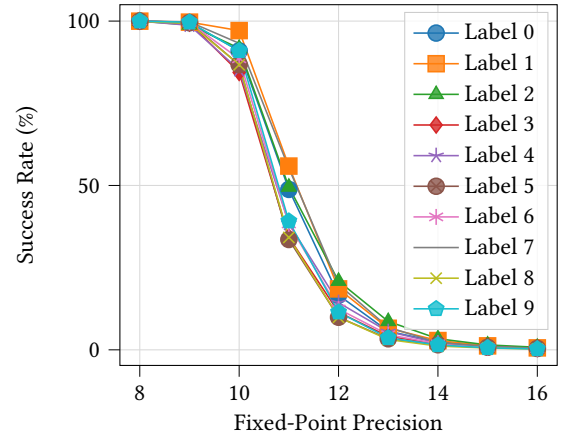


(c) Success rates for misclassifying a target label on a 5-layer DNN on the ECG dataset.

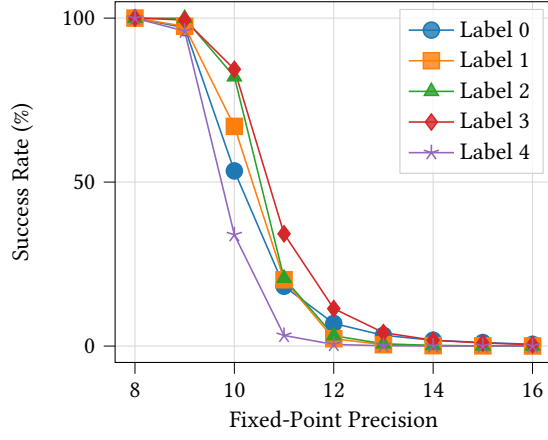
Figure 8: Evaluation of the adversarial example attack on different datasets.



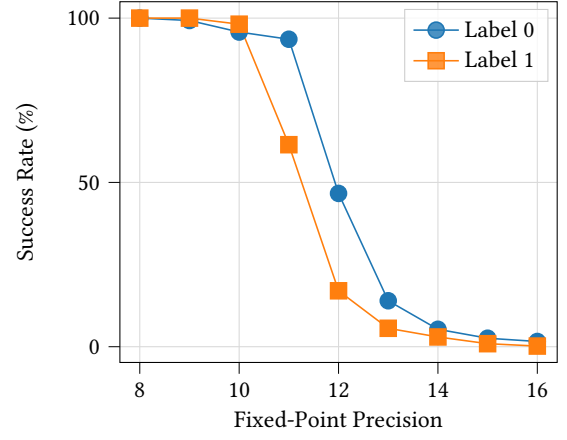
(a) Success rates for misclassifying to a target label on a 5-layer DNN on the MNIST dataset.



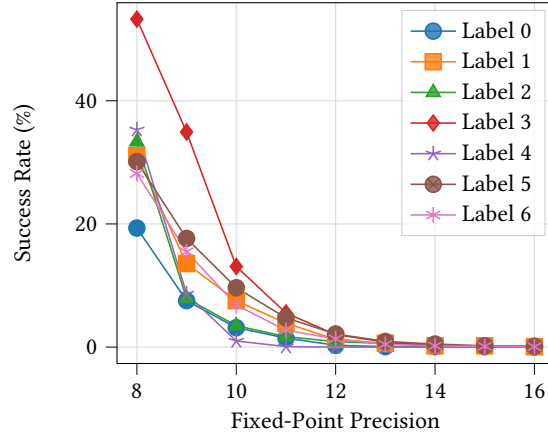
(b) Success rates for misclassifying to a target label on a 5-layer DNN on the CIFAR10 dataset.



(c) Success rates for misclassifying to a target label on a 5-layer DNN on the ECG dataset.



(d) Success rates for misclassifying to a target label on a 5-layer DNN on the VOICE dataset.



(e) Success rates for misclassifying to a target label on a 5-layer DNN on the OBESITY dataset.

Figure 9: Evaluation of the optimisation attack on different datasets.